

# SERRE FRAISIER

Plateforme de Supervision Hydroponique Temps Reel

Guide Technique & Guide de Presentation Jury

|                   |                                          |
|-------------------|------------------------------------------|
| Stack technique : | React + FastAPI + PostgreSQL + WebSocket |
| Protocole :       | WebSocket (RFC 6455) + REST HTTP         |
| Deploiement :     | Docker Compose / Synology NAS            |
| Version :         | 2026 - Projet Academique                 |

Contenu : Architecture | WebSocket | Synology NAS | Guide Jury | Checklist

# 01 Architecture Globale de l'Application

L'application est decoupee en 3 couches independantes. Chaque couche a un role precis et elles communiquent via des protocoles standards (HTTP et WebSocket).



## >> Description des 3 couches principales

|      |                                                                                                                                                                                                                                                                                             |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FRON | <b>FRONTEND - React + Vite (port 3000)</b><br>L'interface utilisateur dans le navigateur web. Elle affiche les graphiques en temps reel (ECharts), la vue 3D (Three.js), les alertes et l'historique. Elle recoit les donnees via WebSocket et fait des requetes REST pour les historiques. |
| BACK | <b>BACKEND - FastAPI Python (port 8000)</b><br>Le serveur central. Il gere deux types de communication : 1) L'API REST pour les requetes (historique, appareils, alertes). 2) Le WebSocket /ws/live qui pousse les nouvelles mesures a tous les clients connectes en meme temps.            |
| BASE | <b>BASE DE DONNEES - PostgreSQL (port 5432)</b><br>Stocke toutes les mesures climatiques (temperature, humidite, CO2, EC, pH...) ainsi que les appareils (ventilateurs) et les alertes. Le simulateur insere une nouvelle mesure toutes les 5 secondes.                                     |

## >> Flux de donnees complet (etape par etape)

```
1. DataSimulator genere une mesure toutes les 5 secondes
2. La mesure est inseree dans PostgreSQL (table internal_data)
3. FastAPI recupere la mesure et appelle ConnectionManager.broadcast()
4. Tous les clients WebSocket connectes recoivent la mise a jour
5. React met a jour les graphiques ECharts en temps reel (< 100 ms)
```

## 02 Protocole WebSocket - Pourquoi et Comment

Le WebSocket est le protocole choisi pour la communication en temps reel. Il ouvre une connexion persistante entre le navigateur et le serveur : le serveur peut envoyer des donnees a tout moment sans que le client ne les redemande.

### >> Comparaison des protocoles

| Protocole          | Fonctionnement                       | Delai    | Bande passante |
|--------------------|--------------------------------------|----------|----------------|
| REST Polling       | Client demande toutes les X secondes | 5 - 30 s | Elevee         |
| WebSocket (choisi) | Connexion ouverte, push instantane   | < 100 ms | Tres faible    |
| SSE                | Serveur pousse, sens unique          | < 200 ms | Faible         |
| MQTT               | Protocole IoT publish/subscribe      | < 50 ms  | Minimale       |

CHOIX RETENU : WebSocket est le meilleur compromis pour ce projet.

Raisons : support natif dans tous les navigateurs, integration directe dans FastAPI, latence < 100 ms, et aucun broker intermediaire necessaire (contrairement a MQTT).

### >> Les 3 avantages clés du WebSocket

#### 1 - Connexion persistante

Une seule "poignee de main" TCP au demarrage. Ensuite, le canal reste ouvert. Pas de reconnexion a chaque mesure. Cela economise du temps de traitement et de la bande passante.

#### 2 - Push serveur instantane

Des que le simulateur ecrit une mesure, le backend appelle broadcast() et TOUS les onglets ouverts recoivent la donnee automatiquement en moins de 100 millisecondes.

#### 3 - Economie de donnees (99%)

En HTTP classique : 800 octets d'en-tetes par requete. En WebSocket : 2 octets. Pour 1 mesure toutes les 5s sur 24h, WebSocket economise plus de 99% de la bande passante.

### >> Message WebSocket envoye par le serveur (format JSON)

```
{
  "type": "internal_update",
  "timestamp": "2026-07-01T14:23:05Z",
  "data": {
    "temperature": 23.4,
    "humidity": 67.2,
    "co2": 850,
    "ec": 1.8,
    "ph": 6.1
  }
}
```

```
}
```

#### >> Code cote frontend (useSensorData.ts - simplifie)

---

```
const ws = new WebSocket("ws://SERVEUR:8000/ws/live")

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data)
  if (msg.type === "internal_update") {
    setTemperature(msg.data.temperature) // graphique se met a jour
    setHumidity(msg.data.humidity)
  }
}

ws.onclose = () => {
  setTimeout(connect, 5000) // reconnexion automatique apres 5s
}
```

### 03 Integration avec le Serveur Synology DS115j

INFORMATION IMPORTANTE : Le DS115j est un NAS ARM avec 512 Mo de RAM.

Il ne supporte pas Docker officiellement.

Pour la presentation jury : garder Docker local + reseau Wi-Fi local.

C'est la solution la plus simple et la plus fiable pour le jour J.

#### >> Option A - Presentation jury (recommandee, la plus simple)

Gardez votre application Docker locale. Toute personne connectee au meme Wi-Fi peut acceder a l'application via votre adresse IP locale :

```
# Etape 1 : Trouver votre adresse IP locale
Windows : ipconfig (chercher "Adresse IPv4")
Linux/Mac: ip addr show (chercher "inet 192.168...")

# Etape 2 : Lancer l'application Docker
docker compose up -d

# Etape 3 : Partager l'URL avec le jury
http://192.168.1.XX:3000 (remplacer XX par votre IP)

# Pour un acces depuis Internet (jury a distance)
ngrok http 3000

# -> genere une URL publique ex: https://abc123.ngrok.io
```

#### >> Option B - Deploiement natif sur le Synology DS115j

Suivez ces 5 etapes dans l'ordre :

|         |                                                                                                                                                                                                                   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Etape 1 | <b>Installer les packages DSM</b><br>Dans DSM -> Centre de paquets : installer Python 3, Web Station, Nginx, PostgreSQL (via SynoCommunity).                                                                      |
| Etape 2 | <b>Activer SSH et uploader le backend</b><br>DSM -> Panneau de configuration -> Terminal -> Activer SSH.<br>Puis: scp -r ./backend/* admin@IP_NAS:/volume1/serre/backend/<br>Et: pip3 install -r requirements.txt |
| Etape 3 | <b>Configurer PostgreSQL sur le NAS</b><br>Se connecter a psql et creer la base :<br>CREATE DATABASE serre_fraisier;<br>Puis mettre a jour DATABASE_URL dans le fichier .env du backend.                          |
| Etape 4 | <b>Compiler et copier le frontend</b><br>Sur votre machine : npm run build (dans le dossier frontend/)<br>Copier le dossier dist/ vers /volume1/web/serre/ sur le NAS.                                            |
|         |                                                                                                                                                                                                                   |

DSM -> Portail de connexion -> Proxy inverse :

/api/\* et /ws/\* -> localhost:8000 (backend)

/\* -> localhost:3000 (frontend)

## 04 Guide de Presentation devant le Jury

Duree recommandee : 15 a 20 minutes. Structurez votre presentation en 5 parties. Commencez par montrer l'application ouverte et fonctionnelle, ensuite expliquez les choix techniques.

|    |                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | <b>Introduction</b> [ 2 min ]<br>Presentez l'objectif : supervision en temps reel d'une serre hydroponique de fraisiers. Montrez le Dashboard ouvert avec les graphiques qui se mettent a jour en direct.                                                   |
| 02 | <b>Architecture</b> [ 3 min ]<br>Dessinez le schema : Simulateur -> PostgreSQL -> FastAPI -> WebSocket -> React. Expliquez le role de chaque brique technique.                                                                                              |
| 03 | <b>Focus WebSocket</b> [ 4 min ]<br>C'est le coeur technique attendu par le jury. Ouvrez 2 onglets du Dashboard cote a cote et montrez la mise a jour simultanee. Expliquez pourquoi WebSocket est meilleur que le polling HTTP classique.                  |
| 04 | <b>Demo des pages</b> [ 5 min ]<br>Parcourez : Dashboard, Historique, Ventilateurs, Alertes, Vue 3D. Activez le mode sombre (plus impressionnant visuellement pour le jury).                                                                                |
| 05 | <b>Integration serveur</b> [ 3 min ]<br>Expliquez comment l'URL WebSocket est configuree via variable d'environnement (VITE_WS_URL) au moment du build Docker. Cela permet de changer de serveur (local, NAS, cloud) sans modifier une seule ligne de code. |

## 05 Questions Frequentes du Jury - Reponses Preparees

### Q : Pourquoi WebSocket et pas MQTT ?

R : MQTT est optimise pour les capteurs physiques contraints (Arduino, ESP32) qui ont peu de memoire. Dans notre cas, le backend Python recoit deja les donnees. WebSocket s'integre directement dans FastAPI et dans les navigateurs sans broker intermediaire.

### Q : Comment gerez-vous la deconnexion reseau ?

R : Le hook useSensorData.ts implemente une reconnexion automatique apres 5 secondes (ws.onclose -> setTimeout(connect, 5000)). En parallele, le polling REST toutes les 30s assure un fallback. L'indicateur EN DIRECT dans la NavBar change de couleur.

### Q : La securite de l'application ?

R : En production : HTTPS pour le frontend et WSS (WebSocket Securise avec TLS) pour les donnees temps reel. Le backend FastAPI genere des tokens JWT. Sur le NAS, le reverse proxy Nginx termine la connexion TLS.

### Q : Comment l'application passe-t-elle du local au serveur ?

R : L'URL du serveur est configuree via des variables d'environnement (VITE\_API\_URL et VITE\_WS\_URL) au moment du build Docker. Il suffit de changer ces valeurs pour pointer vers le NAS ou un serveur cloud sans modifier le code.

### Q : Pourquoi Docker Compose ?

R : Docker garantit que l'application fonctionne de maniere identique sur toutes les machines. Compose orchestre les 3 services (frontend, backend, PostgreSQL) avec leurs dependances. Un seul "docker compose up" demarre tout l'ecosysteme.

### Q : Que se passe-t-il si le serveur tombe ?

R : Le frontend affiche les dernieres donnees en cache et l'indicateur "EN DIRECT" passe en orange "Hors ligne". La reconnexion WebSocket se fait automatiquement toutes les 5 secondes. Le polling REST prend le relais si le WebSocket echoue.



## 06 Checklist - Avant de Passer devant le Jury

Verifiez chaque point dans les 30 minutes qui precedent votre presentation.

### TECHNIQUE

- docker compose up -d lance et stable (verifier avec : docker ps)
- Dashboard visible sur <http://localhost:3000>
- Indicateur "EN DIRECT" vert dans la NavBar (dot vert anime)
- Mode sombre active (plus impressionnant visuellement)
- Vue 3D chargee et visible (attendre 3s le premier chargement)
- Badge rouge sur "Alertes" dans la navigation

### DEMONSTRATION

- Avoir 2 onglets du Dashboard ouverts pour montrer le WebSocket simultanee
- Preparer la navigation : Dashboard -> 3D -> Historique -> Alertes
- Connaître les chiffres clés : 5s cycle, <100ms latence WS, 30s poll REST

### RESEAU

- Connaître votre IP locale (ipconfig sur Windows)
- Verifier que le pare-feu autorise le port 3000
- Tester l'accès depuis un autre appareil sur le meme Wi-Fi
- Si acces Internet necessaire : lancer ngrok http 3000 avant

### PRESENTATION ORALE

- Schema d'architecture prepare (dessin ou papier imprime)
- Difference WebSocket vs REST polling vs MQTT expliquee
- Savoir expliquer VITE\_WS\_URL et les variables d'environnement
- Connaître le nom de chaque page et ce qu'elle montre

**Bonne presentation !**

React + FastAPI + PostgreSQL + WebSocket + Docker